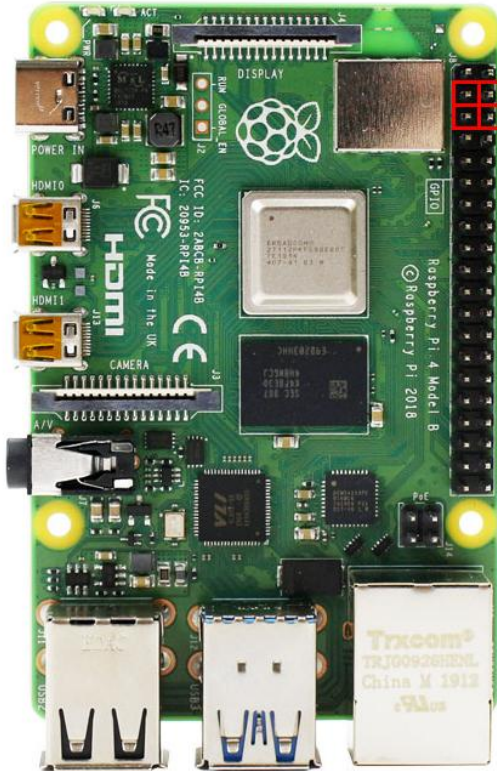


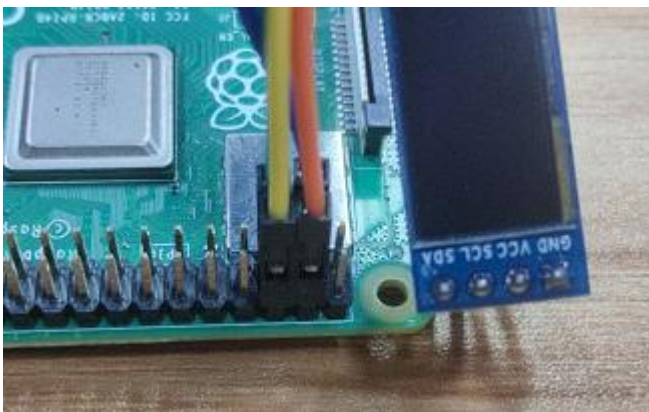
How to Use an OLED Screen on the Raspberry Pi

1. Connecting the OLED Screen to the Raspberry Pi



Pin#	NAME		NAME	Pin#
01	3.3v DC Power		DC Power 5v	02
03	GPIO02 (SDA1 , I ² C)		DC Power 5v	04
05	GPIO03 (SCL1 , I ² C)		Ground	06
07	GPIO04 (GPIO_GCLK)		(TXD0) GPIO14	08
09	Ground		(RXD0) GPIO15	10
11	GPIO17 (GPIO_GEN0)		(GPIO_GEN1) GPIO18	12
13	GPIO27 (GPIO_GEN2)		Ground	14
15	GPIO22 (GPIO_GEN3)		(GPIO_GEN4) GPIO23	16
17	3.3v DC Power		(GPIO_GEN5) GPIO24	18
19	GPIO10 (SPI_MOSI)		Ground	20
21	GPIO09 (SPI_MISO)		(GPIO_GEN6) GPIO25	22
23	GPIO11 (SPI_CLK)		(SPI_CE0_N) GPIO08	24
25	Ground		(SPI_CE1_N) GPIO07	26
27	ID_SD (I ² C ID EEPROM)		(I ² C ID EEPROM) ID_SC	28
29	GPIO05		Ground	30
31	GPIO06		GPIO12	32
33	GPIO13		Ground	34
35	GPIO19		GPIO16	36
37	GPIO26		GPIO20	38
39	Ground		GPIO21	40

The four pins of the OLED screen are connected to pins 3, 4, 5 and 6 of the Raspberry Pi. VCC is connected to pin 4 (5V), GND is connected to pin 6 (Ground), SDA is connected to pin 3 (SDA, I2C), and SCL is connected to pin 5 (SCL, I2C).

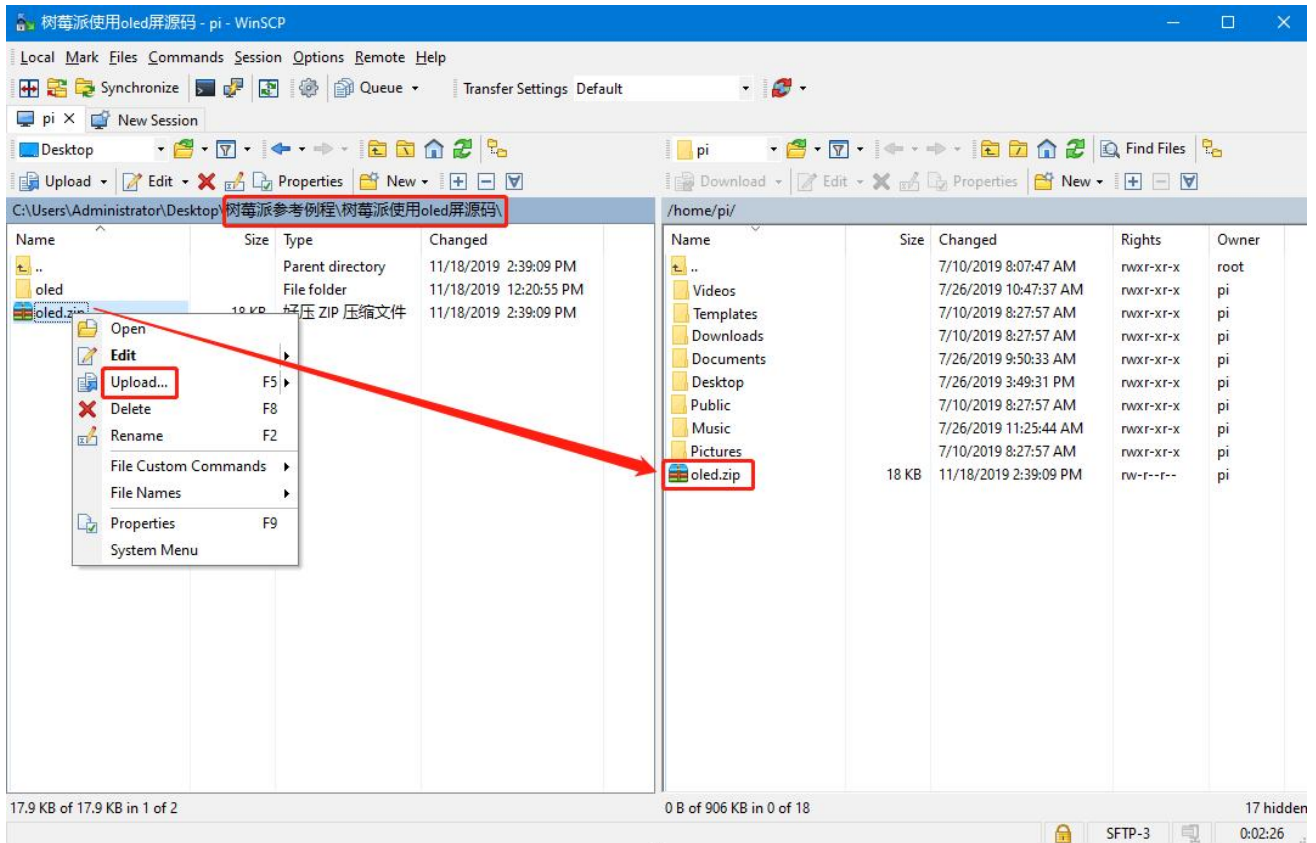


(Refer to the physical wiring diagram)

2. File Transfer

1. Install the WinSCP tool on the computer. After connecting to the Raspberry Pi, transfer the oled.zip archive in the OLED source code folder to the pi directory on the Raspberry Pi. For the method of transferring files between the Raspberry Pi and a Windows computer, please refer to:

<https://www.yahboom.com/build.html?id=2631&cid=308>



2. Unzip the file

Open the terminal on the Raspberry Pi and locate the oled.zip file that was just transferred to the Raspberry Pi.

Enter the following command to unzip it:

unzip oled.zip

```
pi@raspberrypi:~ $ unzip oled.zip
Archive:  oled.zip
  creating: oled/
  inflating: oled/oled
  inflating: oled/oled.c
  inflating: oled/oled_fonts.h
  inflating: oled/ssd1306_i2c.c
  inflating: oled/ssd1306_i2c.h
pi@raspberrypi:~ $
```

3. Compile and Run the Program

1. Enter the folder and view the files in the current folder

cd oled

ls

```
pi@raspberrypi:~ $ cd oled
pi@raspberrypi:~/oled $ ls
oled  oled.c  oled_fonts.h  ssd1306_i2c.c  ssd1306_i2c.h
pi@raspberrypi:~/oled $
```

2. Compile the program file

```
gcc -o oled oled.c ssd1306_i2c.c -lwiringPi
```

```
pi@raspberrypi:~/oled $ gcc -o oled oled.c ssd1306_i2c.c -lwiringPi
ssd1306_i2c.c: In function 'ssd1306_fillRect':
ssd1306_i2c.c:724:3: warning: implicit declaration of function 'swap_values' [-Wimplicit-function-declaration]
    swap_values(x, y);
    ^
pi@raspberrypi:~/oled $ ls
oled oled.c oled_fonts.h ssd1306_i2c.c ssd1306_i2c.h
pi@raspberrypi:~/oled $
```

You can see that the oled file has become an executable file (shown in green). Here, the gcc compiler is invoked, -o indicates the name of the generated file, oled.c and ssd1306_i2c.c are the source programs, and -lwiringPi references the wiringPi library on the Raspberry Pi.

3. Run the program

```
./oled
```

```
pi@raspberrypi:~/oled $ ./oled
init ok!
```

At this point, you can see the Raspberry Pi's CPU usage, CPU temperature, RAM usage, disk usage and IP address displayed on the OLED screen.



4. Code Explanation

1. Import the wiringPi/I2C library, the OLED display library, the file control library, the IP address reading library and the disk reading library.

```
// 导入wiringPi/I2C库
#include <wiringPi.h>
#include <wiringPiI2C.h>

// 导入oled显示屏库
#include "ssd1306_i2c.h"

// 导入文件控制函数库
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/sysinfo.h>
// 读取IP库
#include <ifaddrs.h>
#include <netinet/in.h>
#include <string.h>
#include <arpa/inet.h>
// 读取磁盘库
#include <sys/vfs.h>
#include <unistd.h>

#define TEMP_PATH "/sys/class/thermal/thermal_zone0/temp"
#define MAX_SIZE 32
```

2. Define parameters related to the temperature, system information, disk information, IP, etc.

```
int fd_temp;
double temp = 0;
char buf[MAX_SIZE];

// get system usage / info
struct sysinfo sys_info;
struct statfs disk_info;

struct ifaddrs *ifAddrStruct = NULL;
void *tmpAddrPtr = NULL;
getifaddrs(&ifAddrStruct);
```

3. Initialize the OLED display and output an initialization success message to the terminal.

```
ssd1306_begin(SSD1306_SWITCHCAPVCC, SSD1306_I2C_ADDRESS);
// ssd1306_display(); //show logo
// ssd1306_clearDisplay();
// delay(500);
printf("init ok!\n");
```

4. Read the system information. If it fails, display "sysinfo-Error" on the OLED and wait 0.5 seconds before reading again:

```
// 读取系统信息
if (sysinfo(&sys_info) != 0) // sysinfo(&sys_info) != 0
{
    printf("sysinfo-Error\n");
    ssd1306_clearDisplay();
    char *text = "sysinfo-Error";
    ssd1306_drawString(text);
    ssd1306_display();
    delay(500);
    continue;
}
```

5. Read the CPU usage. The sprintf function is used to concatenate strings.

```
// CPU占用率
char CPUInfo[MAX_SIZE];
unsigned long avgCpuLoad = sys_info.loads[0] / 1000;
sprintf(CPUInfo, "CPU:%ld%%", avgCpuLoad);
```

6. Read the RAM usage. The read data is in units of bytes. For convenient display, it needs to be converted to MB, which can be done by shifting the value right by 20 bits. It can also be written as:
 unsigned long totalRam = sys_info.totalram / 1024 / 1024;

```
// 运行内存占用率，剩余/总内存
char RamInfo[MAX_SIZE];
unsigned long totalRam = sys_info.totalram >> 20;
unsigned long freeRam = sys_info.freeram >> 20;
sprintf(RamInfo, "RAM:%ld/%ld MB", freeRam, totalRam);
```

7. Read the IP address. It can display the IP addresses of both the Ethernet and WiFi connections, prioritizing the Ethernet IP address.

```
// 获取IP地址
char IPInfo[MAX_SIZE];
while (ifAddrStruct != NULL)
{
    if (ifAddrStruct->ifa_addr->sa_family == AF_INET)
    { // check it is IP4 is a valid IP4 Address
        tmpAddrPtr = &((struct sockaddr_in *)ifAddrStruct->ifa_addr)->sin_addr;
        char addressBuffer[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, tmpAddrPtr, addressBuffer, INET_ADDRSTRLEN);

        if (strcmp(ifAddrStruct->ifa_name, "eth0") == 0)
        {
            sprintf(IPInfo, "eth0:IP:%s", addressBuffer);
            break;
        }
        else if (strcmp(ifAddrStruct->ifa_name, "wlan0") == 0)
        {
            sprintf(IPInfo, "wlan0:%s", addressBuffer);
            break;
        }
    }
    ifAddrStruct = ifAddrStruct->ifa_next;
}
}
```

8. Read the temperature.

```
// 读取CPU温度
char CPUTemp[MAX_SIZE];
fd_temp = open(TEMP_PATH, O_RDONLY);
if (fd_temp < 0)
{
    temp = 0;
    printf("failed to open thermal_zone0/temp\n");
}
else
{
    // 读取温度并判断
    if (read(fd_temp, buf, MAX_SIZE) < 0)
    {
        temp = 0;
        printf("fail to read temp\n");
    }
    else
    {
        // 转换为浮点数打印
        temp = atoi(buf) / 1000.0;
        // printf("temp: %.1f\n", temp);
        sprintf(CPUTemp, "Temp: %.1fC", temp);
    }
}
close(fd_temp);
```

9. Read the disk space.

```
// 读取磁盘空间, 剩余/总空间
char DiskInfo[MAX_SIZE];
statfs("/", &disk_info);
unsigned long long totalBlocks = disk_info.f_bsize;
unsigned long long totalSize = totalBlocks * disk_info.f_blocks;
size_t mbTotalsize = totalSize >> 20;
unsigned long long freeDisk = disk_info.f_bfree * totalBlocks;
size_t mbFreedisK = freeDisk >> 20;
sprintf(DiskInfo, "Disk:%ld/%ldMB", mbFreedisK, mbTotalsize);
```

10. Set the content to be displayed on the OLED

```
// 在显示屏上要显示的内容
ssd1306_drawText(0, 0, CPUInfo);
ssd1306_drawText(56, 0, CPUTemp);
ssd1306_drawText(0, 8, RamInfo);
ssd1306_drawText(0, 16, DiskInfo);
ssd1306_drawText(0, 24, IPInfo);

// 刷新显示
ssd1306_display();
```

The `ssd1306_drawText(int x, int y, char *str)` function sets the content to be displayed on the OLED. The first parameter is `x`, which controls the horizontal offset; the second parameter is `y`, which controls the vertical offset; and the third is a string pointer, which is the content to be displayed. Finally, the `ssd1306_display()` function must be run to refresh the display.